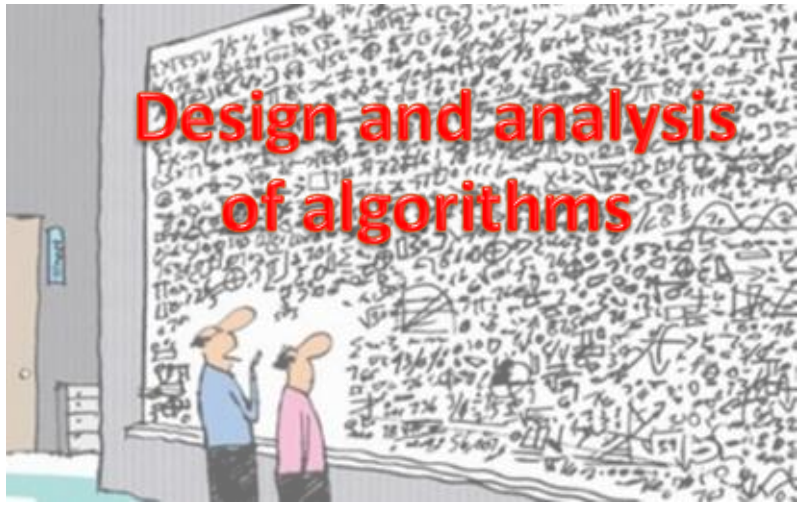


- Shortest paths
- Bellman-Ford algorithm

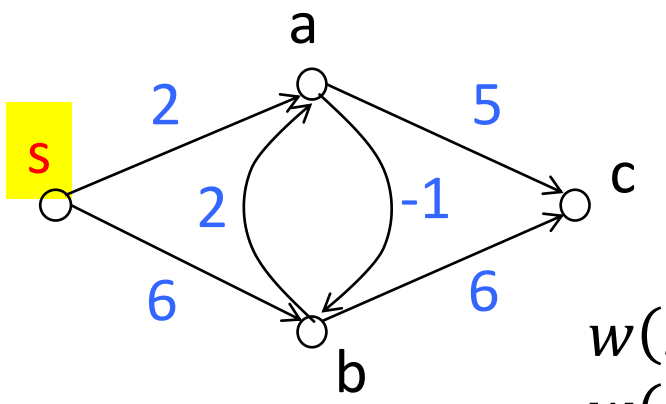
Objective

- Define the **single source** shortest path problem.
- Relaxations and Bellman equations.
- The Bellman-Ford algorithm.
- DPV Chapter 4.
- CLRS 24.0-24.3.

Weighted paths in graphs

- Given:
  - A directed graph $G = (V, E)$.
  - A weight function w: $E \rightarrow R$.
  - A path $p =< v_0, v_1, \ldots, v_k >$.
  - The weight of a path p: $w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i)$.

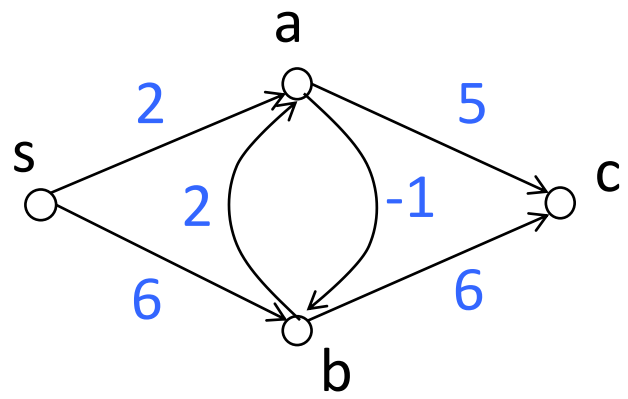
***Simple* path**: a path that does not repeat nodes.
Cycle: path $v_0, v_1, \ldots, v_k, v_0$



$$w(sabc) = 2 + (-1) + 6 = 7$$
$$w(sababc) = 2 + (-1) + 2 + (-1) + 6 = 8.$$

Shortest-paths problems

- $\delta(u, v)$: *shortest distance* from u to v: weight (length) of shortest-path from u to v.
- $$\delta(u,v) = \begin{cases} \min_p \{w(p) : u \xrightarrow{p} v\} & \text{if there is at least one path } p \text{ from } u \text{ to } v, \\ \infty & \text{if there is no path.} \end{cases}$$
- This shortest distance is realized along a set of shortest paths $SP(u, v)$.



$\delta(s, c) = 7,$
$SP(s, c) = \{(sabc), (sac)\}.$

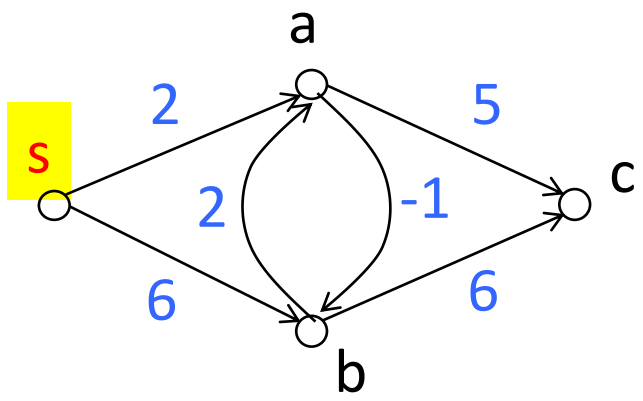
$\delta(u, v)$ always exists and is unique. But there may be many SPs that correspond to $\delta(u, v)$.

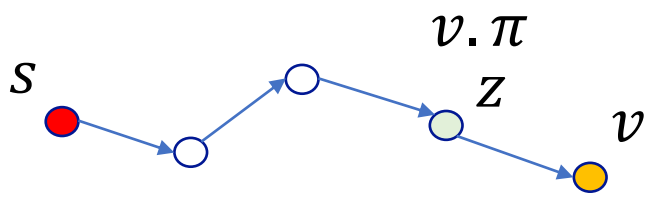
Fix the starting node

SPs to a given node: just reverse the graph.

- We fix the starting node s, i.e., no need to mention it each time.
- Each node v likes to know: (i) his shortest path distance $\delta(v)$, (ii) his predecessor node $v.\pi$ along one such a shortest path.
- Shortest distance of node b: $\delta(b) \triangleq \delta(s,b) = 1, v.\pi = a.\ SP(b) = \{(sab)\}$.
- $\delta(c) \triangleq \delta(s,c) = 7$, through nodes a or b. $SP(c) = \{(sabc), (sac)\}$.

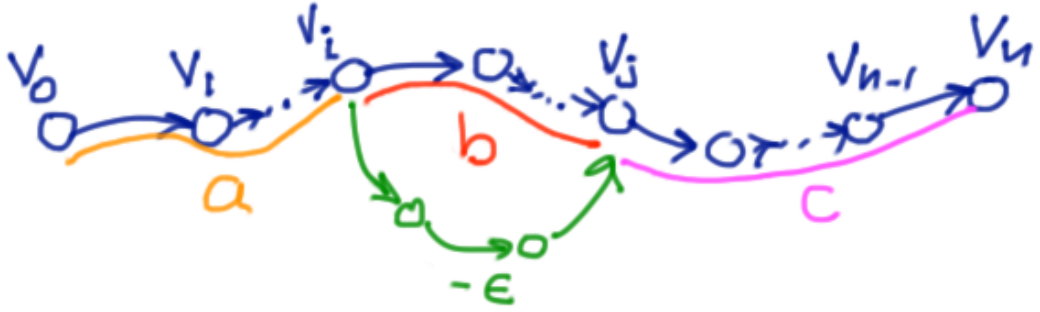
$$c.\pi = a$$





Important property of optimal paths

- Sub-paths of shortest paths are also shortest paths. It follows that
- If v_j on optimal path from v_0 to v_n: $\delta(v_0, v_n) = \delta(v_0, v_j) + \delta(v_j, v_n)$.

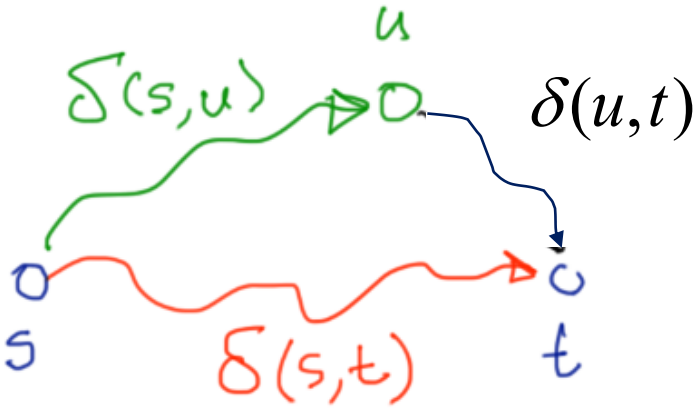


- If the sub-path v_i to v_j is not optimal, then by finding a shorter path from v_i to v_j we can strictly improve the original path.

Triangle inequality

shortest paths
= straight lines

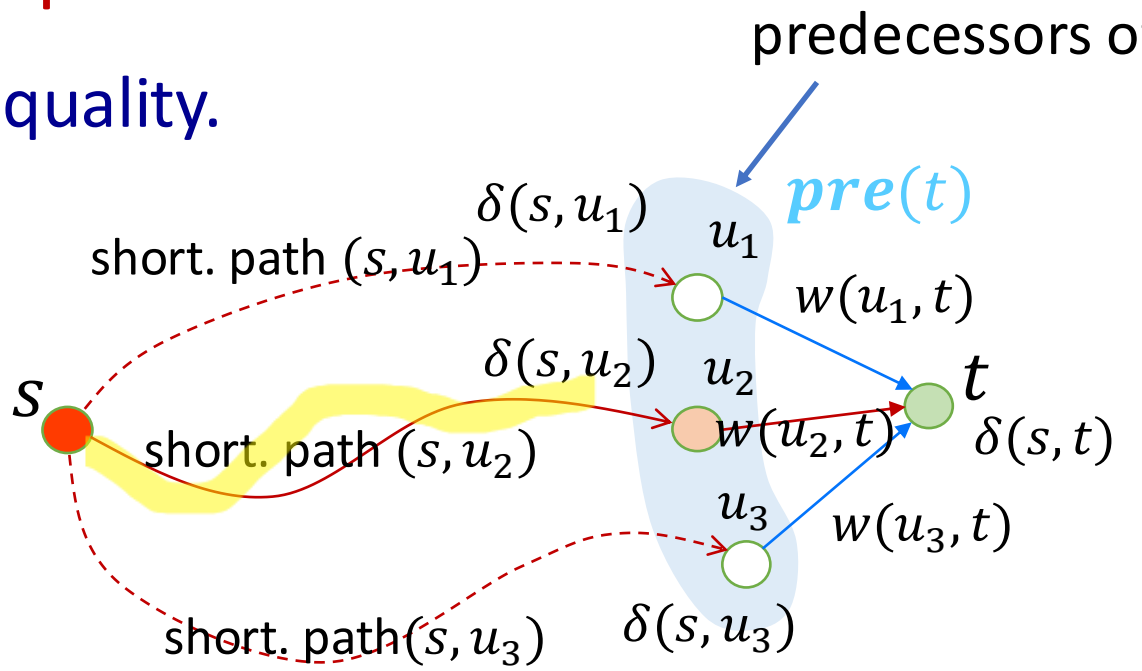
- Generalizes the case of a Euclidean graph.



- In general: $\delta(s,t) \leq \delta(s,u) + \delta(u,t)$.
- If u **not** on shortest path from s to t: $\delta(s,t) < \delta(s,u) + \delta(u,t)$.
- u **is on** shortest path from s to t iff $\delta(s,t) = \delta(s,u) + \delta(u,t)$.

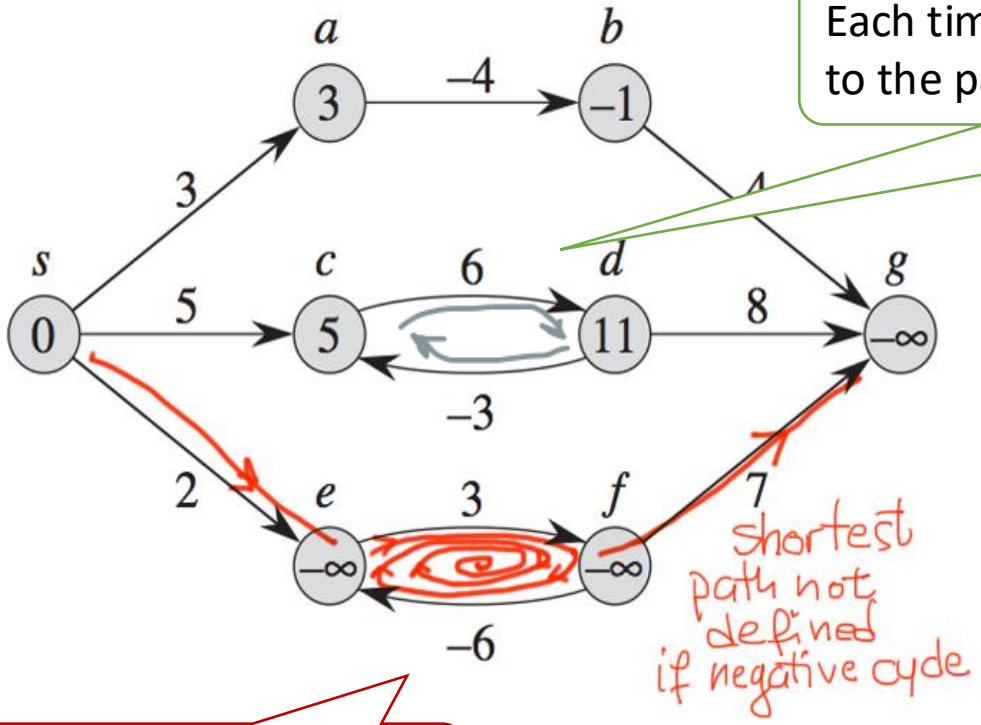
Triangle inequality for predecessors

- A special case of the triangle inequality.



- $\delta(s,t) = \min_{u \in \boldsymbol{pre}(t)} \{\delta(s,u) + w(u,t)\}$.
- If u not on shortest path from s to t: $\delta(s,t) < \delta(s,u) + w(u,t)$.
- u is on shortest path from s to t iff $\delta(s,t) = \delta(s,u) + w(u,t)$.

Source of difficulty: Negative weight edges



Each time we use the positive cycle we add +3 to the path, hence we don't like to repeat the cycle.

Repeating the negative cycle adds -3 to path.

Conclusion: negative weight edges create problems only if they generate negative cycles.
No negative cycles ⇒ shortest paths are always "**simple**" (no repetition of nodes).

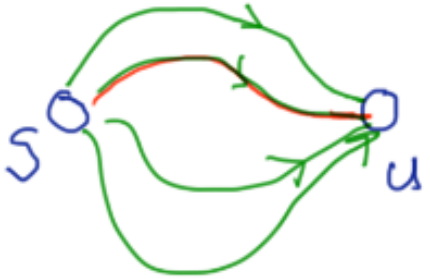
Graphs with no negative cycles

- Assume DGs with **no negative cycles.**
- Initial state $= s$.

hence shortest distance is achieved over paths that are **simple**.

Proposition: $\delta(s,u)$ is well-defined and

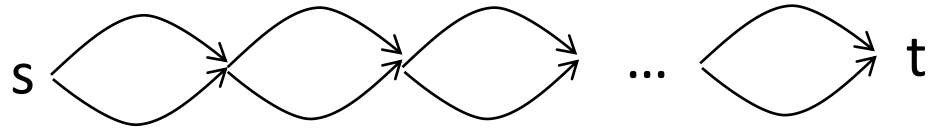
$$\delta(s,u) = \begin{cases} < \infty & \text{for reachable nodes,} \\ = \infty & \text{for unreachable nodes.} \end{cases}$$



Proof:

- We lose nothing to consider shortest paths = simple paths.
- From $s \to u$ there is a finite set of simple paths, pick one with the smallest weight.
- But there can be exponentially many simple paths.

of simple paths can be exponential!



Equations for shortest distance

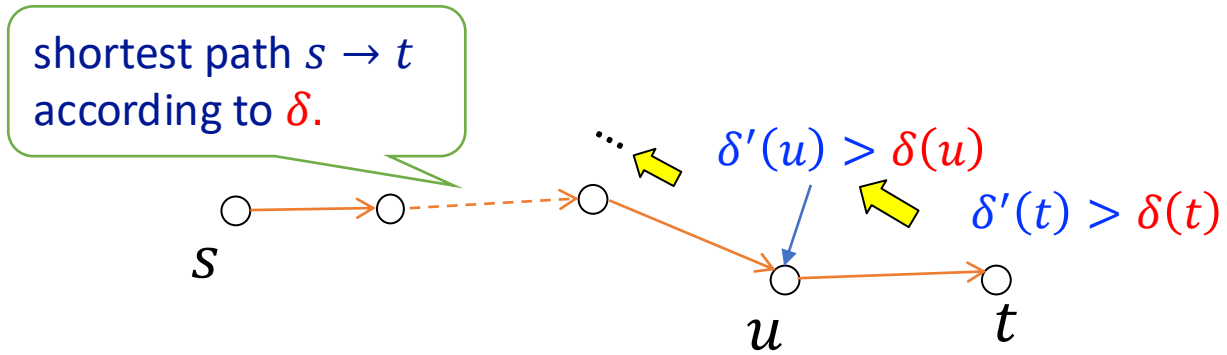
Proposition: (characterization of $\delta(.)$)

- $\delta(s,t)$ satisfies the Bellman equations:

$$\delta(s,t) = \min_{u \in pre(t)} \{\delta(s,u) + w(u,t)\},$$

$$\delta(s,s) = 0.$$

- The solution $\delta(.)$ of the equations is **unique**.
  - Defines also $t.\pi$ (non-unique).
- Proof: assume another solution δ', $\delta'(t) > \delta(t)$.



$\delta(t) = \delta(u) + w_{u,v}$ (from Bellman)

$\delta'(t) \leq \delta'(u) + w_{u,v}$

$\delta'(t) > \delta(t)$

$0 < \delta'(t) - \delta(t) \leq \delta'(u) - \delta(u) \Rightarrow$

$\delta'(u) > \delta(u)$.

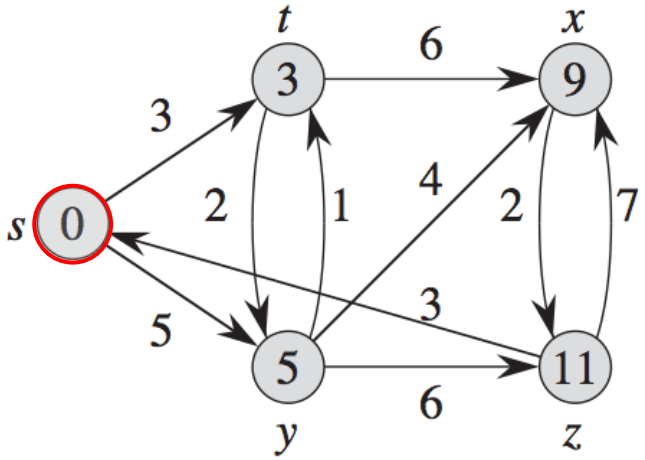
Continue backwards on path until s,

$\Rightarrow \delta'(s) > \delta(s)$.

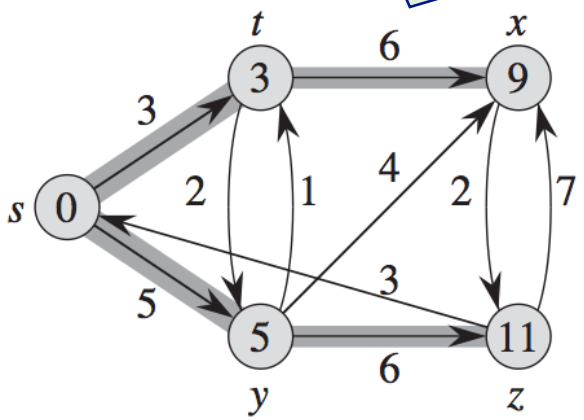
contradiction

Shortest path trees (positive weights)

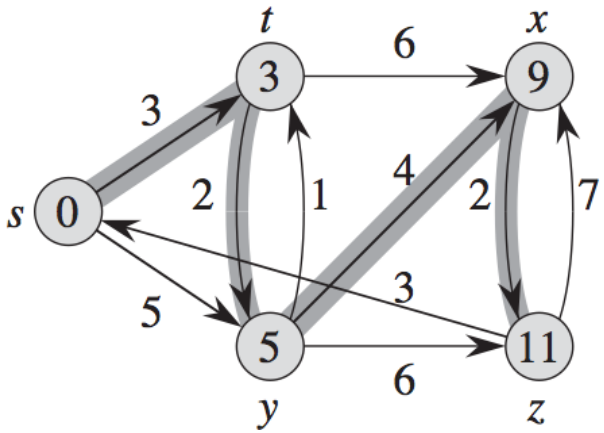
- Solve the Bellman equations for $\delta(t), \forall t$; follow pointers $t.\pi$ backwards to s.
- This defines an SPT.
- Why?



$\delta_x = \delta_t + w_{tx}$
$9 = 3 + 6$



$\delta_x \leq \delta_y + w_{yx}$: $9 \leq 5 + 4$
$\delta_x \leq \delta_z + w_{zx}$: $9 \leq 11 + 7$



If we knew an *SPT*, we have solved our problem of finding $\delta(.)$.
It gives us the order of solving the Bellman equations: on each level going away from s
How to use this interesting observation?

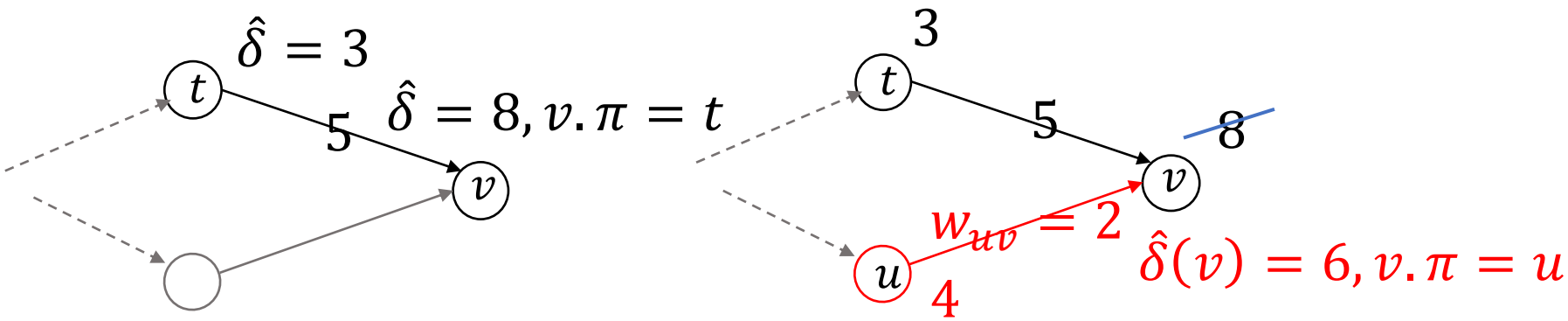
Using relaxations

Improving our shortest distance estimates step by step

Improving the estimates

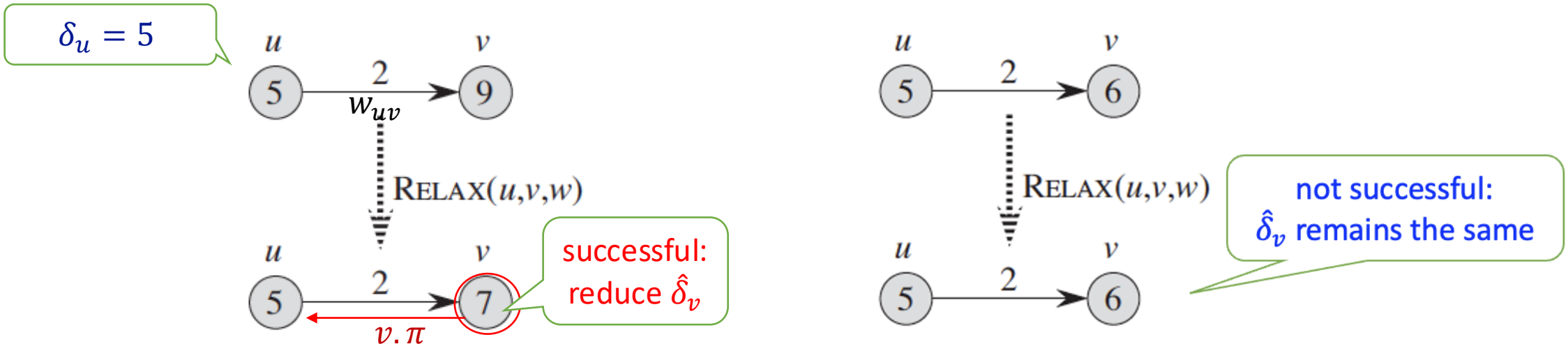
- At any time, nodes store their current estimates $\hat{\delta}(v)$ of their shortest distance and advertise it to their neighbors.
- Same as "a guarantee that v can be reached from s with no more than $\hat{\delta}(v)$ cost".
- There must be always a *justification*:
  - For some predecessor node $t \to v$, we must have $\hat{\delta}_t + w_{tv} = \hat{\delta}_v$
- If for some edge (u, v), $\hat{\delta}_u + w_{uv} < \hat{\delta}_v$, then v can strictly improve its guarantee $\hat{\delta}'_v = \hat{\delta}_u + w_{uv}$ by using u as its predecessor, $v.\pi = u$.

new value



Relaxations ("update operations" in DPV)

- Relaxing edge (u, v):
  - improve the estimate $\hat{\delta}_v$ of v through (u, v): $\hat{\delta}'_v = \min\{\hat{\delta}_v, \hat{\delta}_u + w_{uv}\}$.
- If initially all estimates $\hat{\delta}_u$ are "safe", i.e., $\hat{\delta}_u \geq \delta_u \ \forall u \in V$, then after any relaxation they remain safe: *cannot underestimate the true* δ.



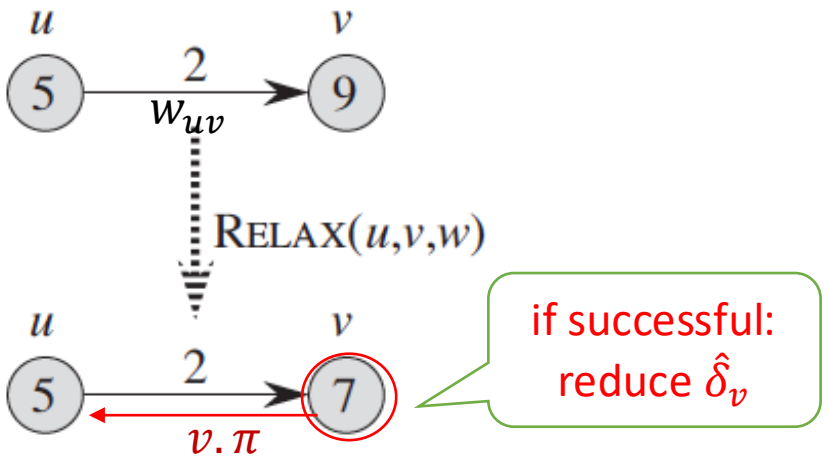
If successful, make it a tree edge

update $v.\pi = u$

If not successful, leave things the way they are

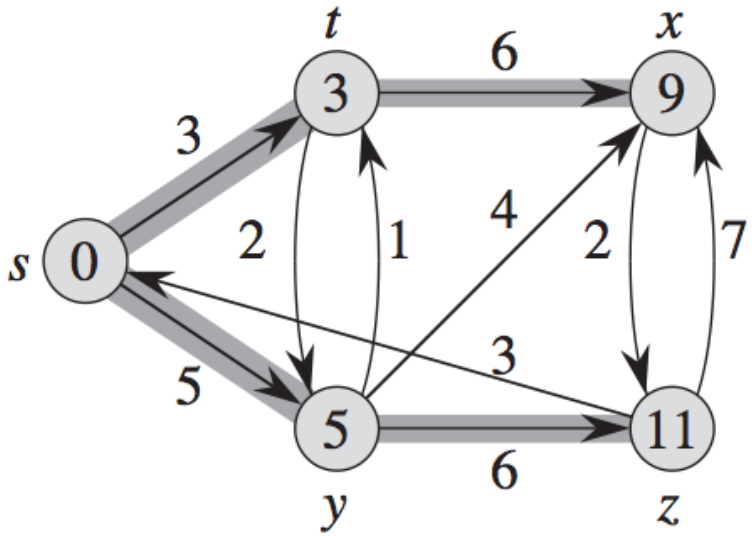
Relaxations converge to an SPT

- If successful relaxations still possible, Bellman equations are not satisfied.
- If no more successful relaxations possible ⇒ Bellman equ. satisfied ⇒ SPT.
- *Construct the SPT:* Just follow backwards the $v.\pi$ pointers for each node.

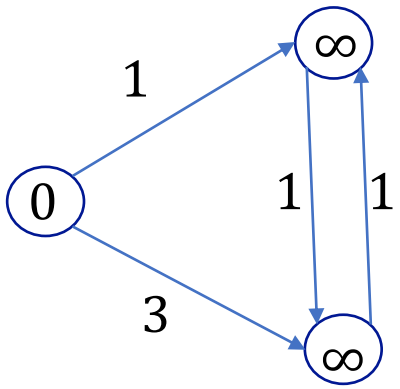


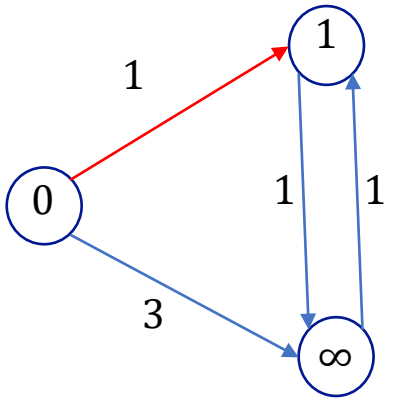
If successful, make it a tree edge

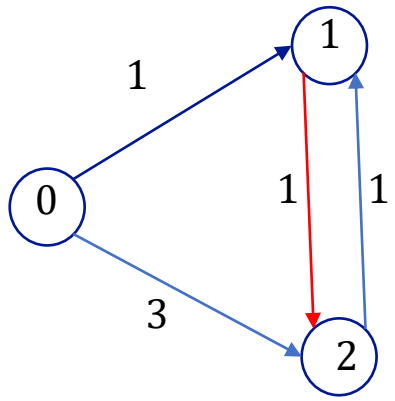
update $v.\pi = u$

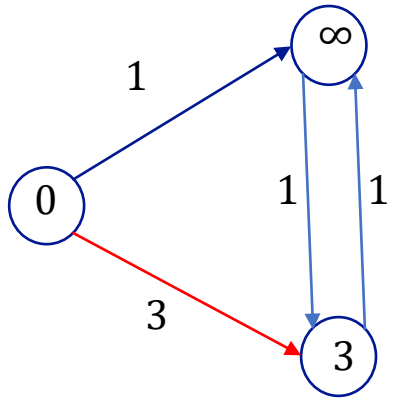


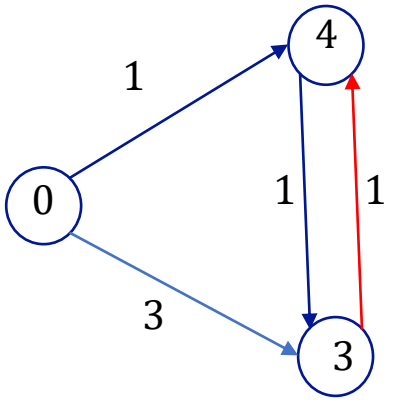
Example

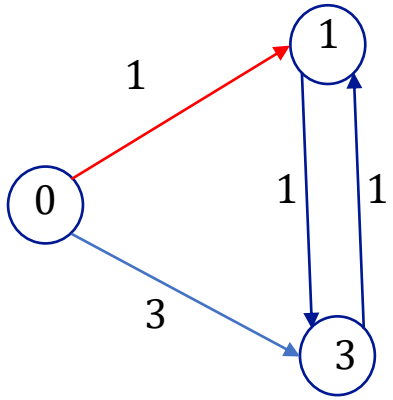


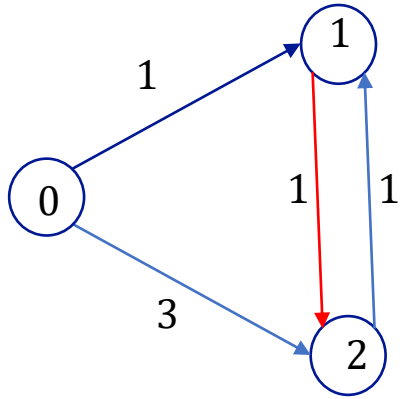


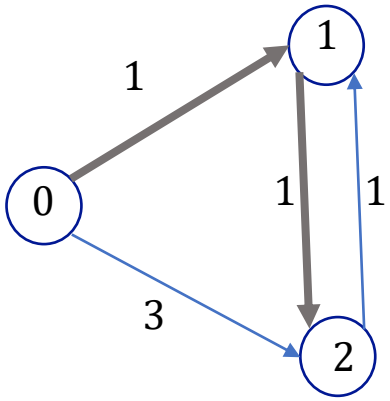












Algorithm: Keep relaxing edges successfully until no more possible.
⇒ **Shortest path tree**

is this a good algorithm?

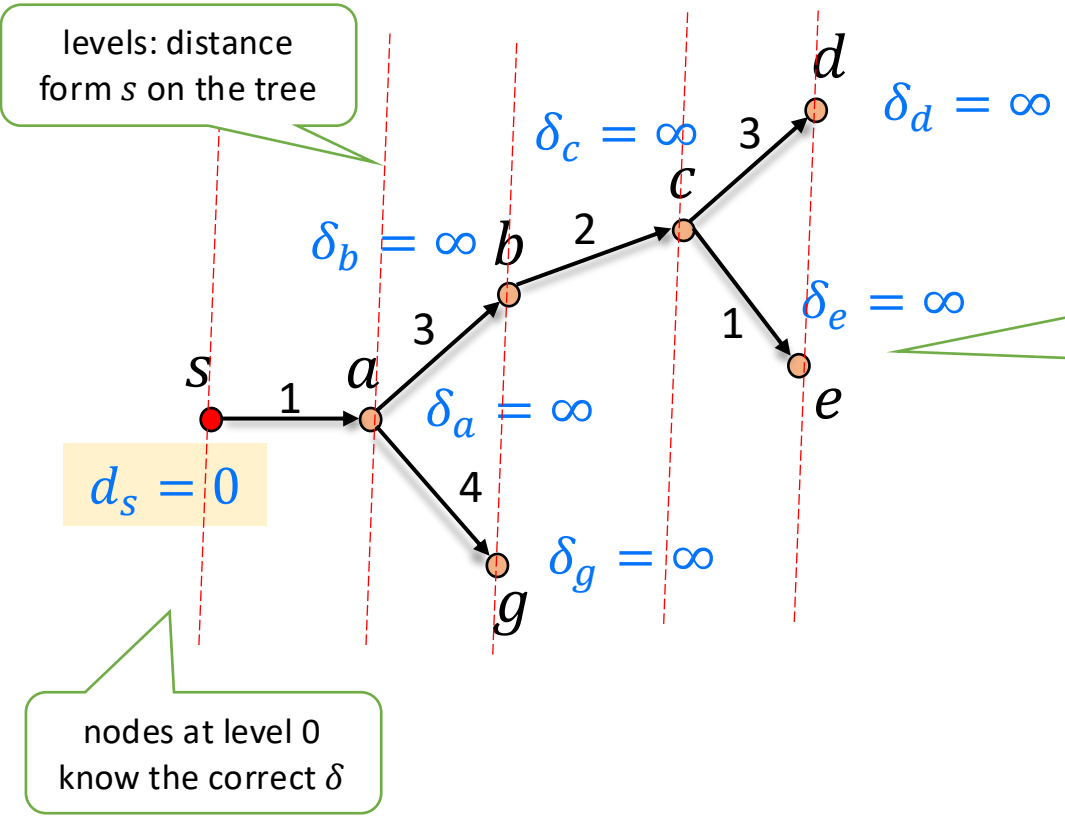
Idea of an algorithm

- Iterative algorithm: we start with some safe estimate $\hat{\delta}(.) = \infty$ of $\delta(.)$ and keep improving it by relaxations until we get to the actual $\delta(.)$.
- Do until nothing changes:
  - Relax one all edges in this graph (in any order)
- What will happen at the end?

Let's check!

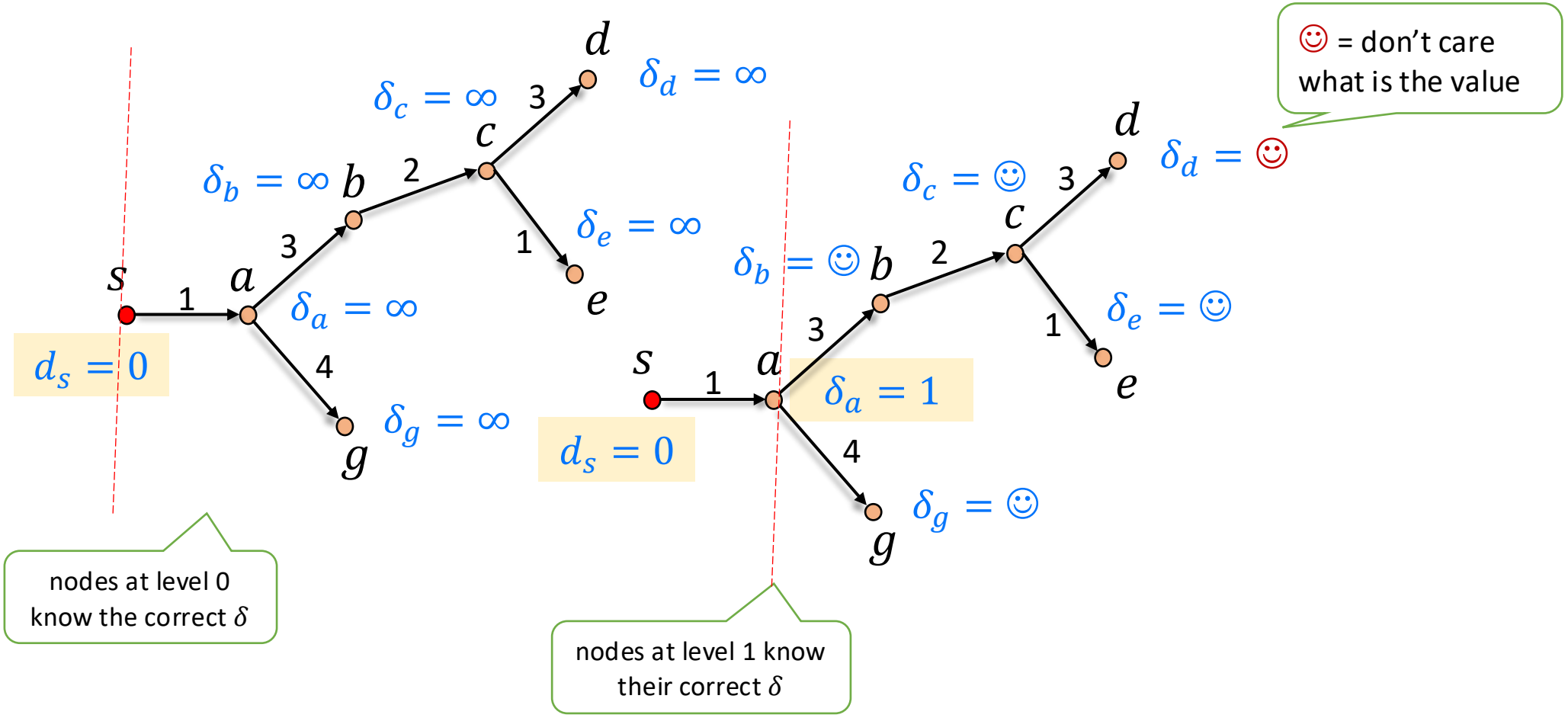
Don't worry how we know the SPT

- Assume that we know an SPT. Initially all estimates are $\hat{\delta} = \infty$.

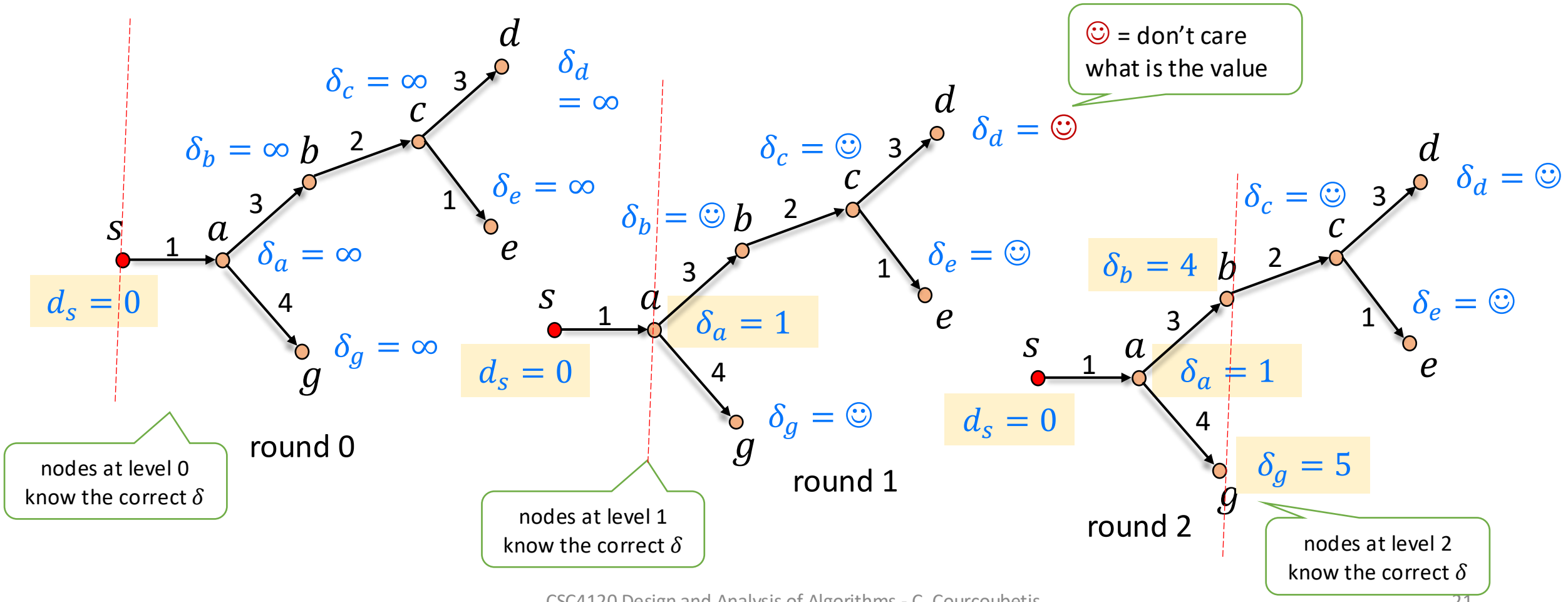


We show **only** the edges of the the SPT. There are **more edges** that we don't show here.

Relax all edges x 1

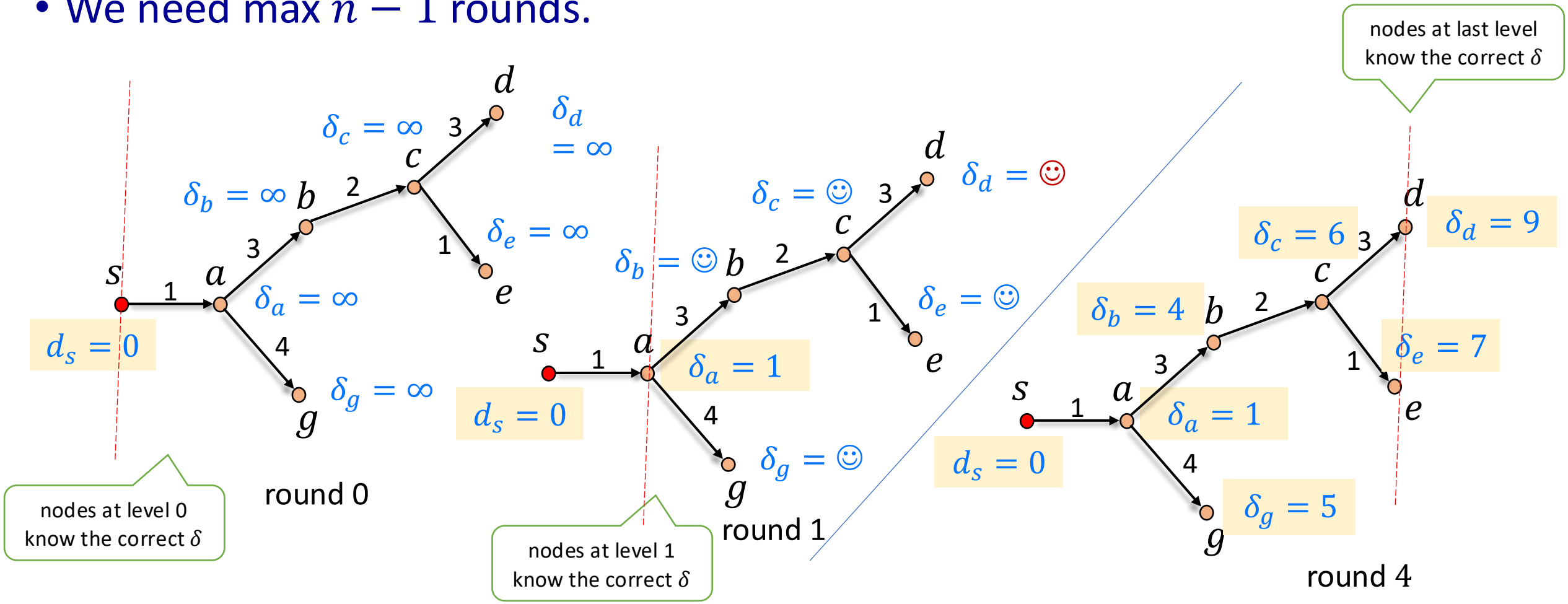


Relax all edges x 2



Relax all edges x 4

- All the nodes on the SP tree will know their actual SP distance.
- We need max $n - 1$ rounds.

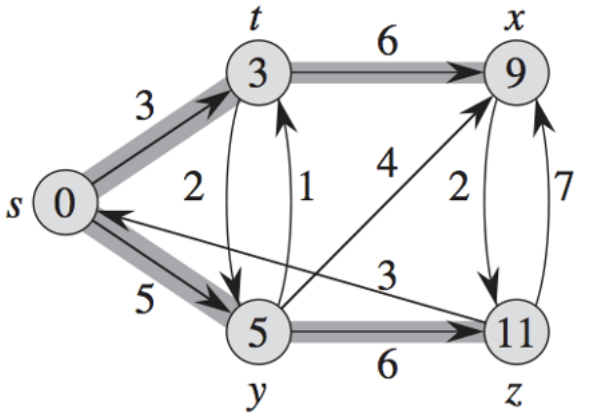


We got an algorithm!

- (a) Start with a **safe estimate** of shortest distances for all nodes.
- (b) Perform $|V| - 1$ times "relax all edges of G": to compute δ_u for all nodes $u \in V$.
- OK! but only if no negative cycles (since then, unique shortest distances).
- How to test for negative cycles?
- (c) Relax all edges **once more**: if the values of $\hat{\delta}_u, u \in V$ don't change, we are OK.

These 3 steps constitute the Bellman-Ford algorithm

Bellman-Ford algorithm




```
for v in V:
      v.d = ∞
      v.π = nil
s.d = 0
do n-1 times:
     for each edge (u,v) in E:
          relax (u,v)

for each edge (u,v) in E:
     if v.d > u.d+ w(u,v):
          report a negative cycle & return
report no negative cycles
```

initialization
$\Theta(|V|)$

$n = |V|$

main loop
$\Theta(|V||E|)$

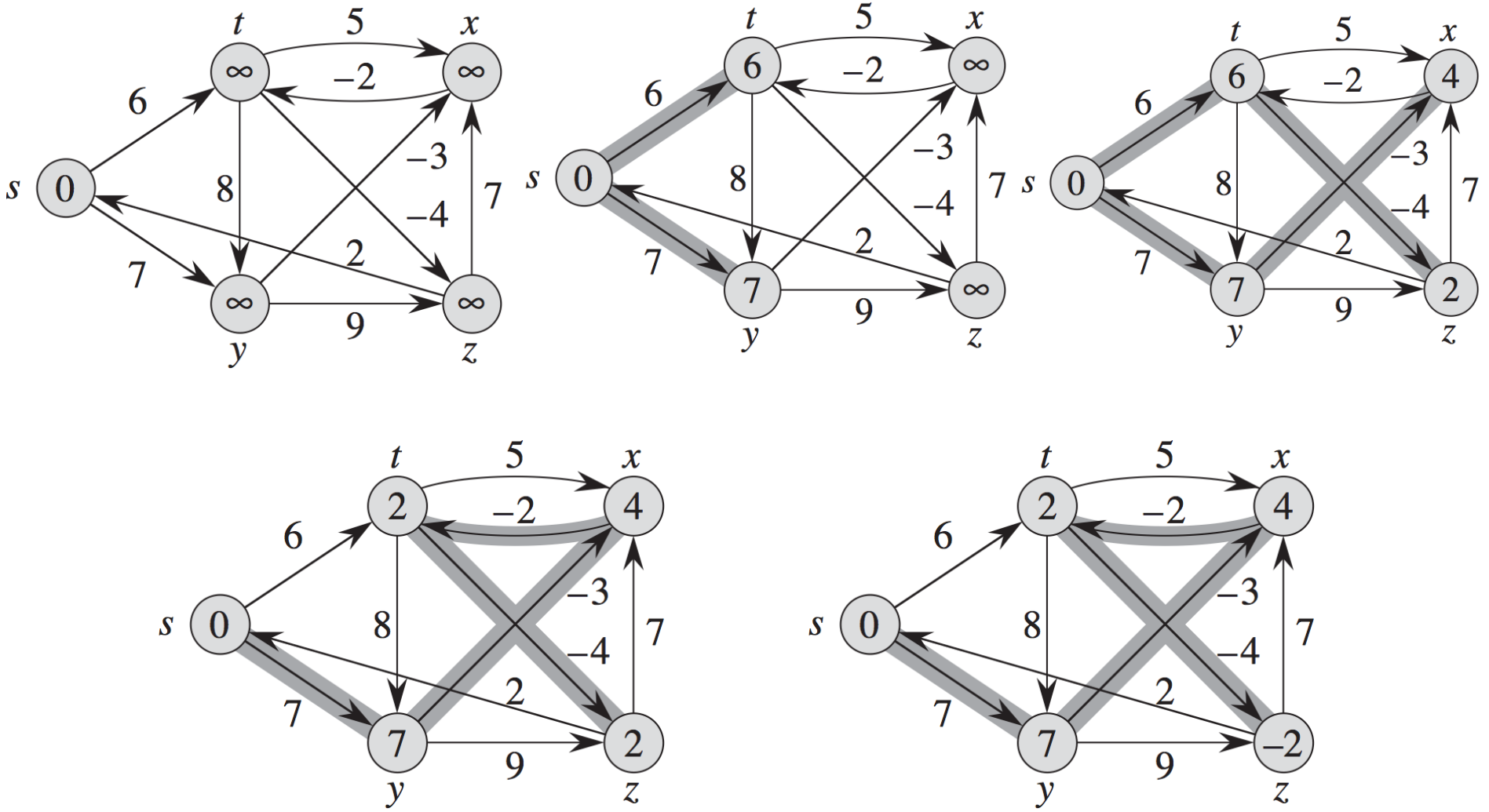
$\hat{\delta}_v$ would change if one more step.

negative cycle detection
$\Theta(|E|)$

Complexity: $\Theta(|V||E|)$
$\Theta(|V|^3)$: dense graphs
$\Theta(|V|^2)$: sparse graphs

Property: if no negative cycles, then at termination for all v: $v.d = \delta(s, v)$.

Bellman-Ford example

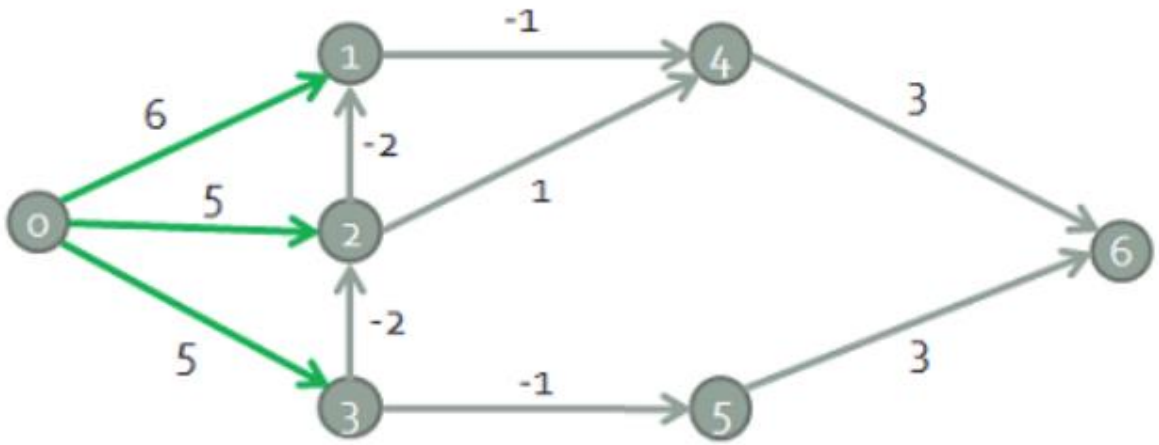


Conclusions

- Shortest paths from a single source exist when there are no negative loops; they form a shortest-path tree.
- The "value function" δ (weight of shortest path) at each node satisfy the Bellman equations (triangle inequality).
- δ satisfies a set of inequalities with at least one equality per node and can be solved by "relaxations" starting from some conservative estimate.
- If we could guess the shortest path tree, then $|V| - 1$ relaxations could lead to the solution of δ.
- Bellman-Ford algorithm: plays safe and relaxes all m edges $|V| - 1$ times, with complexity $\Theta(nm)$.

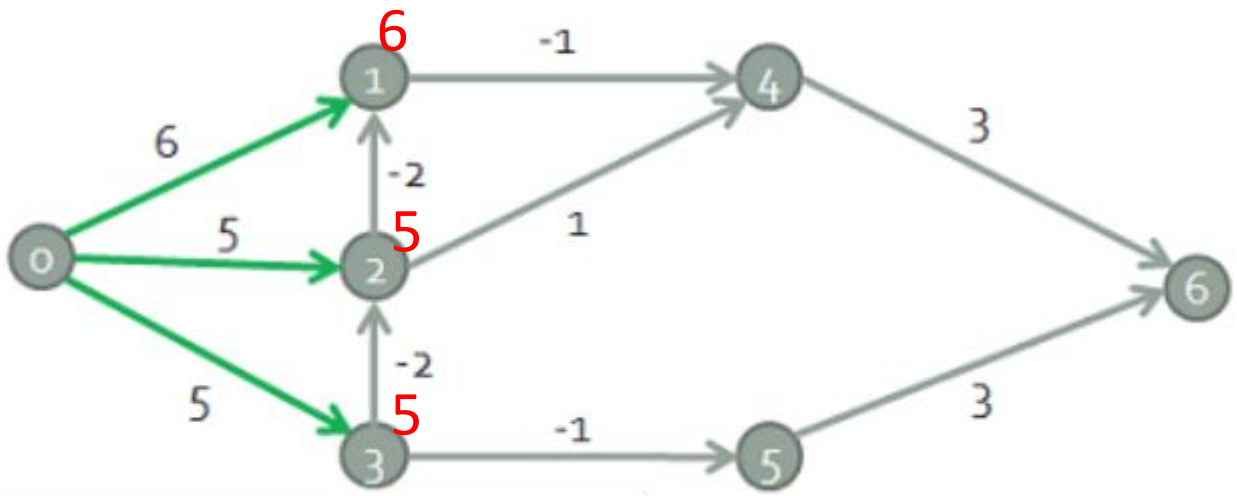
Example 2

don’t worry about the different color of edges



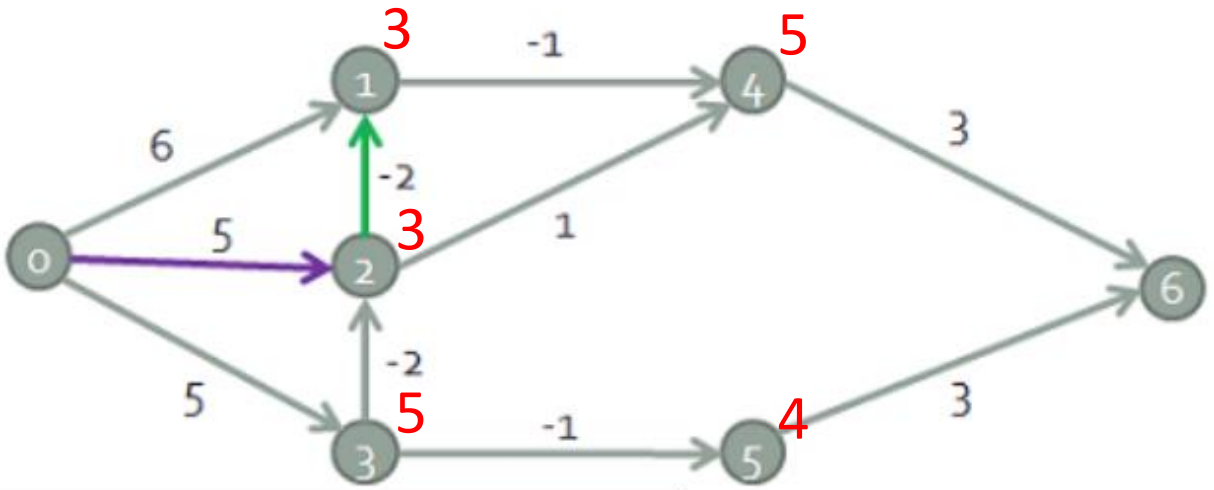
Find the shortest path from node 0

Example 2 Answer



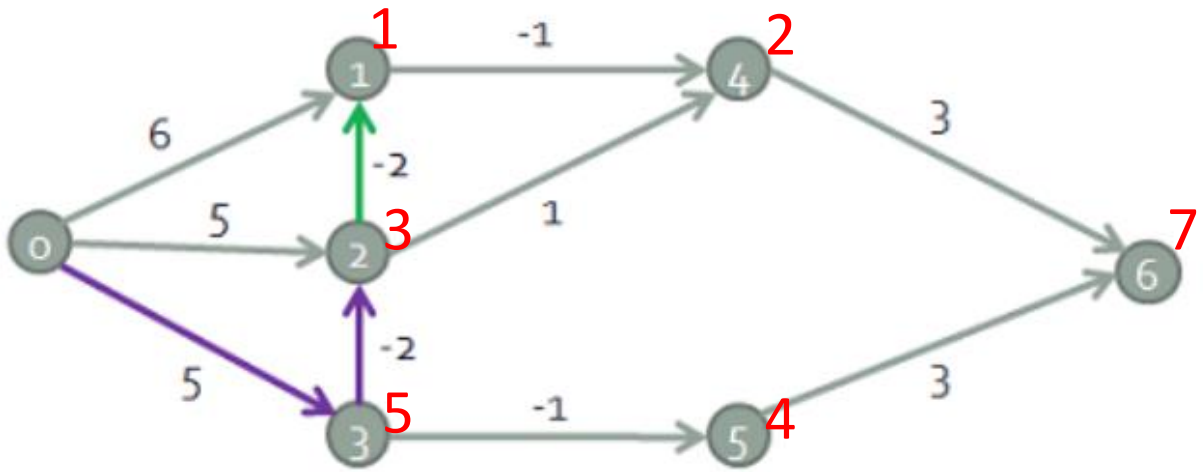
k	1	2	3	4	5	6
1	6	5	5	∞	∞	∞
2						
3						
4						
5						
6						

Example 2 Answer



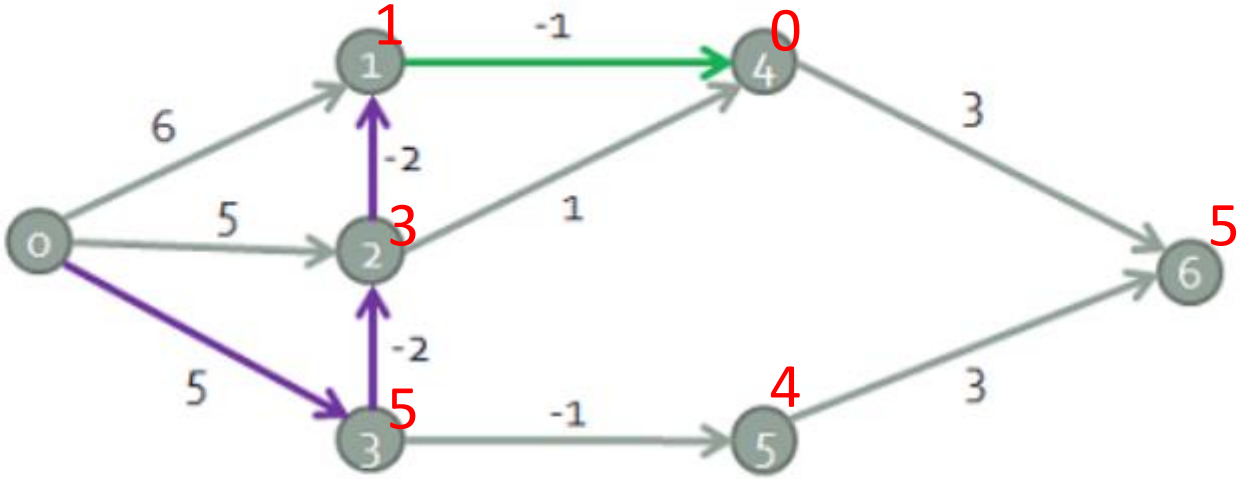
k	1	2	3	4	5	6
1	6	5	5	∞	∞	∞
2	3	3	5	5	4	∞
3						
4						
5						
6						

Example 2 Answer



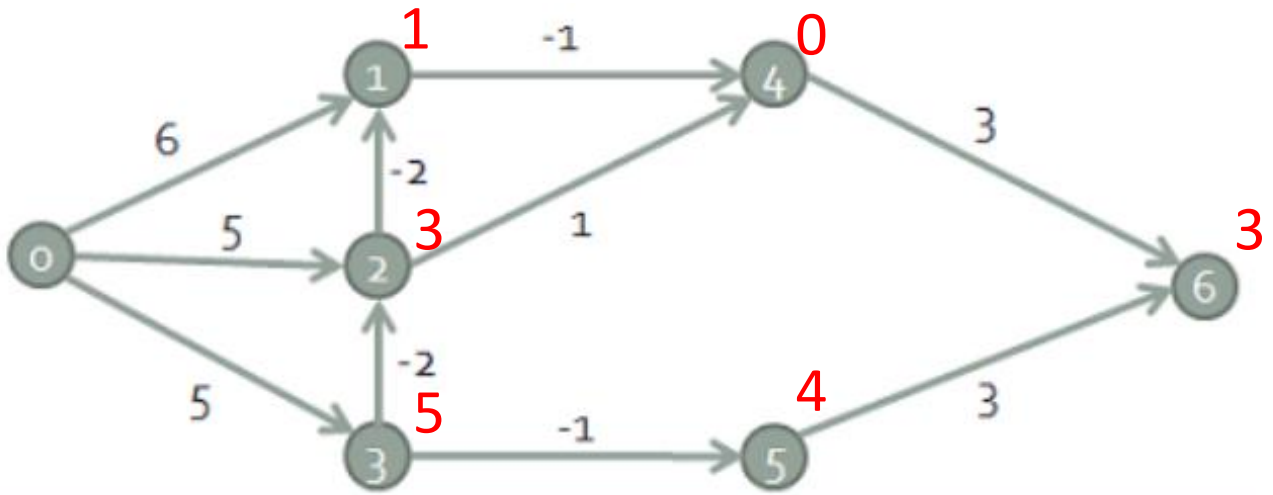
k	1	2	3	4	5	6
1	6	5	5	∞	∞	∞
2	3	3	5	5	4	∞
3	1	3	5	2	4	7
4						
5						
6						

Example 2 Answer



k	1	2	3	4	5	6
1	6	5	5	∞	∞	∞
2	3	3	5	5	4	∞
3	1	3	5	2	4	7
4	1	3	5	0	4	5
5						
6						

Example 2 Answer



k	1	2	3	4	5	6
1	6	5	5	∞	∞	∞
2	3	3	5	5	4	∞
3	1	3	5	2	4	7
4	1	3	5	0	4	5
5	1	3	5	0	4	3
6	1	3	5	0	4	3